

AWS Web Application Firewall (WAF)

Project Title: AWS Web Application Firewall (WAF) Implementation

Objective

The objective of this project is to implement **AWS Web Application Firewall (WAF)** to protect a web application from common web-based attacks and unwanted traffic. AWS WAF allows us to create rules that control which HTTP/HTTPS requests are allowed or blocked before they reach the application.

AWS Services Used

- **AWS WAF**
- **Application Load Balancer (ALB)**
- **Amazon EC2**
- **Amazon CloudWatch** (*for monitoring, if enabled*)

Implementation Steps

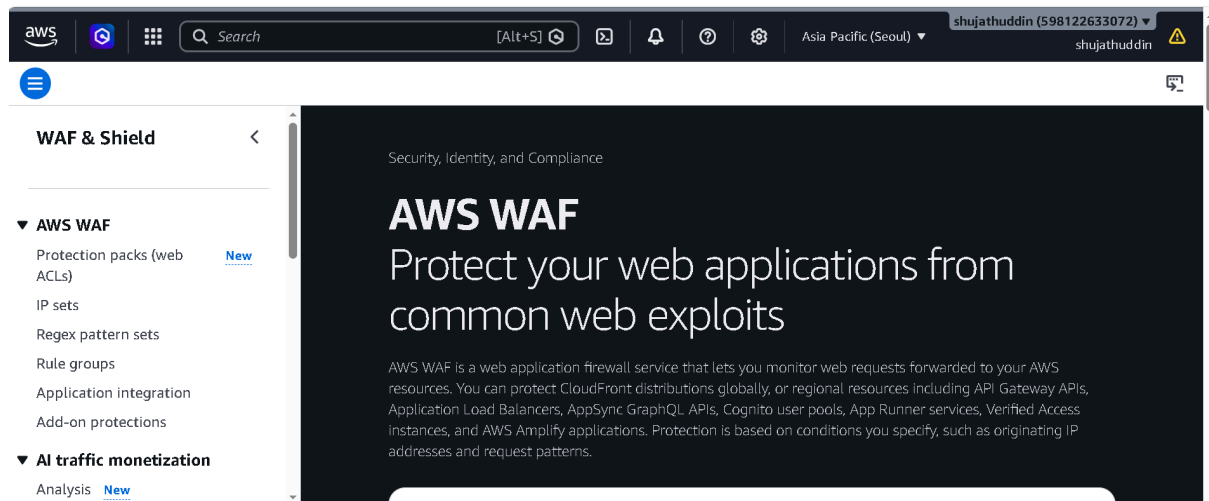
1. **Created an Application Load Balancer**
 - Configured an internet-facing Application Load Balancer.
 - Registered the EC2 web server as a target.
 - Configured HTTP/HTTPS listener rules.
2. **Created a Web ACL**
 - Opened **AWS WAF & Shield** from the AWS Management Console.
 - Created a Web ACL for the required AWS Region.
 - Selected the appropriate AWS resource, such as the Application Load Balancer.
3. **Configured WAF Rules**
 - Added rules to inspect incoming web requests.
 - Configured rules to block suspicious or unwanted traffic.
 - Used AWS Managed Rules to protect against common web application vulnerabilities.
4. **Configured IP-Based Filtering**
 - Created an IP set for specific IP addresses when required.
 - Configured WAF rules to allow or block traffic based on source IP addresses.
5. **Configured Rate-Based Protection**
 - Added a rate-based rule to detect excessive requests from a single IP address.
 - Configured the rule to temporarily block IP addresses exceeding the defined request limit.
6. **Associated WAF with ALB**
 - Associated the Web ACL with the Application Load Balancer.
 - Incoming HTTP/HTTPS requests were inspected by AWS WAF before being forwarded to the application.

7. Tested the Configuration

- Accessed the application through the ALB DNS name.
- Tested normal web requests to verify that legitimate traffic was allowed.
- Tested configured WAF rules to verify that matching requests were blocked.

8. Monitored WAF Activity

- Reviewed WAF metrics and sampled requests.
- Monitored allowed and blocked requests.
- Used CloudWatch metrics/logging when enabled for additional visibility.



AWS Systems Manager Parameter Store

Project Title: AWS Systems Manager Parameter Store Implementation

Objective

The objective of this project is to use **AWS Systems Manager Parameter Store** to securely store and manage application configuration data, environment variables, and sensitive parameters without hardcoding them directly into application code or configuration files.

AWS Services Used

- **AWS Systems Manager Parameter Store**
- **Amazon EC2**
- **AWS IAM**

Implementation Steps

1. **Opened Parameter Store**
 - Navigated to **AWS Systems Manager** → **Parameter Store**.
 - Selected the appropriate AWS Region.
2. **Created a Parameter**
 - Created a new parameter with a suitable name.

- Example:
 - Parameter name: `/myapp/database/host`
 - Type: `String`
 - Value: Database endpoint.
- 3. **Created Secure Parameters**
 - Created sensitive parameters using **SecureString**.
 - Example:
 - `/myapp/database/password`
 - Type: `SecureString`
 - Value: Database password.
 - SecureString parameters are encrypted using AWS Key Management Service (AWS KMS).
- 4. **Configured IAM Permissions**
 - Created or used an IAM role for the EC2 instance.
 - Granted the required permissions to read parameters from Parameter Store.
 - Used permissions such as:
 - `ssm:GetParameter`
 - `ssm:GetParameters`
- 5. **Retrieved Parameters from EC2**
 - Connected to the EC2 instance.
 - Used the AWS CLI to retrieve stored parameters.

Used Parameters in an Application

- Retrieved configuration values from Parameter Store instead of storing them directly in application code.
- This helped separate application configuration from application logic.

Tested Access

- Verified that the EC2 instance could retrieve authorized parameters.
- Tested IAM permissions to ensure unauthorized access was prevented.

